

Time Series Forecasting Using Recurrent Neural Networks: A Comparative Analysis of Elman, Jordan, and Multi-Recurrent Architectures

M Tashreeq Hartogh

27163245

Department of Computer Science

Stellenbosch University

Stellenbosch, South Africa

<https://github.com/calebwidogast96-art/RNN-Forecasting>

Abstract—This report presents a comprehensive comparative analysis of three recurrent neural network (RNN) architectures for time series forecasting: Elman RNNs (Simple RNN), Jordan RNNs, and Multi-Recurrent RNNs. I evaluate these models on five benchmark datasets (Electricity, ETTh1, COVID-19, Weather, and Traffic) that exhibit varying degrees of stationarity and complexity. Using a thorough empirical methodology incorporating time series cross-validation, data preprocessing, and systematic hyperparameter tuning, I assess model performance across different lookback windows and hidden unit configurations. The results show that Multi-Recurrent RNNs with multi-scale temporal dependencies achieved superior performance on most datasets, with Mean Absolute Error (MAE) values consistently lower than traditional Elman RNNs. This study provides insights into the effectiveness of different RNN architectures for practical time series forecasting applications and discusses the trade-offs between model complexity and predictive accuracy.

Index Terms—time series forecasting, recurrent neural networks, Elman RNN, Jordan RNN, multi-recurrent RNN, deep learning

I. INTRODUCTION

Time series forecasting is a fundamental problem in machine learning with applications in finance, weather prediction, energy demand forecasting, epidemiology, and traffic management. Unlike traditional supervised learning where samples are independent and identically distributed (i.i.d.), time series data contains temporal dependencies and correlated structures that require specialized modeling approaches.

Recurrent Neural Networks (RNNs) have emerged as a powerful class of models for time series tasks due to their ability to maintain hidden state across time steps, effectively capturing temporal patterns and dependencies. However, the variety of RNN architectures available presents practitioners with important design choices that can significantly affect forecasting accuracy.

A. Motivation and Research Questions

The main aim for this study is to systematically compare and evaluate different RNN architectures designed to capture temporal information at varying scales. Specifically, I investigate the following research questions:

- 1) How do different RNN architectures (Elman, Jordan, Multi-Recurrent) perform across diverse time series datasets with different statistical properties?
- 2) What is the impact of lookback window size and hidden unit count on model performance?
- 3) Which architectural features (e.g., output feedback, multi-scale delays) provide the most benefit for time series forecasting?
- 4) How do these architectural innovations compare to established baselines (GRU, LSTM)?

B. Contributions

This work makes the following contributions:

- An empirical evaluation of RNN variants using thorough time series validation methodology.
- Implementation of a Multi-Recurrent RNN architecture with configurable multi-scale temporal dependencies.
- Comprehensive analysis of hyperparameter effects (lookback, hidden units) across multiple datasets.
- Clear guidelines for practitioners on architecture selection for time series forecasting.

C. Report Structure

The remainder of this report is organized as follows:

Section II provides theoretical background on RNN architectures and time series methodologies.

Section III describes the five datasets used in my analysis.

Section IV details data preprocessing and implementation specifics.

Section V describes my empirical process and experimental methodology.

Section VI presents and discusses the results. Finally,

Section VII concludes with recommendations for practitioners.

II. BACKGROUND

A. Recurrent Neural Networks

Recurrent Neural Networks are a class of neural networks designed to process sequential data by maintaining a hidden

state h_t that is updated at each time step. The fundamental recurrence relation is:

$$h_t = \sigma(W_{hh}h_{t-1} + W_{xh}x_t + b_h) \quad (1)$$

where x_t is the input at time step t , W_{hh} and W_{xh} are weight matrices, b_h is the bias term, and σ is an activation function (typically tanh or ReLU).

1) *Elman RNN (Simple RNN)*: The Elman RNN, also known as a Simple RNN, is the basic RNN architecture where the hidden state from the previous time step is fed back as input to the current step. The output at time t is:

$$y_t = W_{hy}h_t + b_y \quad (2)$$

While seemingly simple, Elman RNNs suffer from vanishing and exploding gradient problems during backpropagation through time (BPTT), limiting their ability to capture long-term dependencies.

2) *Jordan RNN*: The Jordan RNN improves upon the Elman architecture by replacing the hidden state feedback with output feedback. The recurrence is:

$$h_t = \sigma(W_{hx}x_t + W_{yh}y_{t-1} + b_h) \quad (3)$$

where y_{t-1} is the output from the previous time step, not the hidden state. This design has the advantage of a clearer interpretation of what is being fed back (actual predictions) and can reduce the gradient problem by providing a shorter error propagation path.

3) *Multi-Recurrent RNN*: The Multi-Recurrent RNN extends the standard architecture by using connections from multiple previous hidden states at different delay intervals:

$$h_t = \sigma\left(W_{hx}x_t + \sum_{d \in \text{delays}} W_d h_{t-d} + b_h\right) \quad (4)$$

This architecture explicitly models temporal dependencies at multiple scales, allowing the network to learn patterns at different frequencies. For instance, with delays (1, 6, 12), the network can capture immediate effects, weekly patterns, and monthly patterns simultaneously.

B. Preventing Overfitting and Underfitting

1) *Regularization Strategies*: To prevent overfitting while avoiding underfitting, I implement:

- **Early Stopping**: Monitor validation loss and stop training when it increases for 5 consecutive epochs, restoring the best weights. This prevents overfitting while ensuring the model has trained sufficiently.
- **Validation Set Monitoring**: Use separate validation and test sets to distinguish between underfitting and overfitting.

2) *Capacity Control*: Model capacity is controlled through:

- Systematic variation of hidden units (8 to 512) to find the optimal complexity.
- Careful selection of lookback window to avoid both too-short contexts (underfitting) and too-long contexts (overfitting and computational inefficiency).

C. Performance Metrics

1) *Mean Squared Error (MSE)*: The loss function used for training is:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (5)$$

MSE penalizes larger errors more heavily, making it sensitive to outliers.

2) *Mean Absolute Error (MAE)*: Reported as the primary evaluation metric:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad (6)$$

MAE is scale-dependent but interpretable in the original data units and more robust to outliers than MSE. To account for the scale of the data, min-max scaling was applied. Thus all MAE values throughout the rest of this report are scaled.

III. DATASETS

We selected five diverse benchmark datasets that exhibit different temporal characteristics, allowing comprehensive evaluation of RNN architectures.

A. Dataset Overview

TABLE I: Summary of datasets

Dataset	Variables	Granularity	Length	Stationarity
Electricity	321	15 minutes	26,304 obs	Non-stationary
ETTh1	6	1 hour	17,420 obs	Non-stationary
Weather	21	10 minutes	52,560 obs	Non-stationary
Traffic	430	1 hour	17,544 obs	Non-stationary
COVID-19	55	1 day	365 obs	Non-stationary

B. Electricity Load Dataset

Source: UCI Machine Learning Repository (ElectricityLoadDiagrams20112014).

The electricity dataset contains electricity consumption data from 321 individual clients measured at 15-minute intervals over several years. The target variable is the origin (OT) electricity consumption.

Characteristics:

- High-dimensional multivariate time series (320 features + 1 target).
- Clear daily and weekly seasonality patterns.
- Non-stationary due to trend and seasonal components.

C. ETT Dataset (Electricity Transformer Temperature)

Source: GitHub ETDataset Repository.

The Electricity Transformer Temperature (ETT) dataset comprises measurements from electric power transformers including six load variables: HUFL (High Use Ful Load), HULL (High Use Low Load), MUFL (Middle Use Ful Load), MULL (Middle Use Low Load), LUFL (Low Use Ful Load), and LULL (Low Use Low Load).

Characteristics:

- Moderate dimensionality (6 variables).
- Hourly granularity captures daily heating cycles.
- Clear periodicity and trend components.
- Relatively clean data with minimal missing values.

D. Weather Dataset

Source: Max Planck Institute for Biogeochemistry.

This dataset contains 21 meteorological indicators collected at 10-minute intervals, including temperature, humidity, pressure, radiation, and precipitation measurements.

Characteristics:

- Moderate dimensionality (21 variables).
- High-frequency sampling (10-minute intervals).
- Strong seasonal patterns (daily and annual cycles).
- Non-stationary trend due to seasonal changes.

E. Traffic Dataset

Source: Traffic Data Repository.

The traffic dataset comprises hourly traffic speed measurements from 430 San Francisco freeway car lanes.

Characteristics:

- High-dimensional (430 variables representing different free lanes).
- Clear weekly patterns (weekday vs. weekend).
- Multiple peaks during rush hours.
- Non-stationary due to urban development and seasonal variations.

F. COVID-19 Dataset

Source: Johns Hopkins University CSSE.

Contains COVID-19 hospitalization data for the U.S. state of California (CA), provided by Johns Hopkins University.

Characteristics:

- Moderate dimensionality (55 series).
- Daily sampling (lowest frequency in our collection).
- Non-stationary with multiple structural breaks (lock-downs, policy changes, variants).
- Highly irregular patterns not amenable to simple seasonality.

IV. IMPLEMENTATION

A. Data Preprocessing

All datasets undergo identical preprocessing to ensure fair comparison.

1) *Train/Validation/Test Split:* Chronological splitting is applied to respect temporal ordering:

- **Training Set:** First 70% of chronological data.
- **Validation Set:** Next 15% (between 70% and 85%).
- **Test Set:** Final 15% (after 85%).

This split simulates realistic scenarios where models are trained on historical data and evaluated on unseen future data.

2) *Normalization:* Min-Max scaling is applied to each variable independently:

$$x_{\text{scaled}} = \frac{x - \min(x)}{\max(x) - \min(x)} \quad (7)$$

where min and max are computed on the training set only, then applied to validation and test sets to prevent information leakage.

3) *Sequence Creation:* Time series are converted into supervised learning problems using a sliding window approach. For a given lookback window L , we create sequences:

$$\{(x_{t-L}, \dots, x_{t-1}, y_t) : t = L + 1, \dots, T\} \quad (8)$$

where x_t represents the multivariate features at time t and y_t is the target variable at time t .

B. Model Architecture

1) *Network Configuration:* All RNN variants are configured as follows:

- **Input Layer:** Accepts sequences of shape (batch_size, lookback, num_features).
- **RNN Layer:** Variable hidden units from {8, 16, 32, 64, 128, 256, 512}.
- **Output Layer:** Single neuron with linear activation for regression.
- **Activation Function:** Hyperbolic tangent (tanh) for hidden units.

2) *Optimization:*

- **Optimizer:** Adam (Adaptive Moment Estimation).
- **Learning Rate:** 10^{-3} (default).
- **Loss Function:** Mean Squared Error (MSE).
- **Batch Size:** 32 samples.
- **Maximum Epochs:** 50.
- **Early Stopping:** Patience of 5 epochs on validation loss.

C. Experimental Framework

Implementation is conducted in Python 3.x using TensorFlow 2.x with custom layer implementations for Jordan and Multi-Recurrent RNNs to enable precise control over recurrent connections. (See `custom_layers.py` in the GitHub repository)

- **JordanRNN:** Custom layer implementing output feedback mechanism.
- **MultiRecurrentRNN:** Custom layer implementing multi-scale delays.
- **SimpleRNN:** TensorFlow's built-in Elman RNN layer.

V. EMPIRICAL PROCESS

A. Experimental Design

The empirical process follows a systematic grid search over hyperparameters, each architecture with differing amount of hidden units at differing lookback ranges while maintaining temporal integrity.

1) *Hyperparameter Grid*: For each dataset and architecture, we systematically vary:

TABLE II: Hyperparameter search grid

Architecture	Hidden Units
Simple RNN	8, 16, 32, 64, 128, 256, 512
Jordan RNN	8, 16, 32, 64, 128, 256
Multi-Recurrent RNN	64 (fixed)

Where Multi-Recurrent RNN was tested across the following delays: (1), (1, 2), (1, 2, 3), (1, 2, 3, 4), (1, 6, 12), (1, 3, 6, 12), (1, 6, 12, 24)

Dataset-Specific Lookback Ranges:

- **Electricity**: 6, 12, 18, 24, 30, 36, 42, 48 (6-hour to 48-hour intervals).
- **ETTh1**: 6, 12, 18, 24, 30, 36, 42, 48, 54, 60 (6-hour to 60-hour).
- **Weather**: 12, 24, 36 (2-hour to 6-hour windows at 10-min granularity).
- **Traffic**: 12, 24, 36, 48 (12-hour to 48-hour windows).
- **COVID-19**: 1 through 21 (1 to 21 days).

B. Experimental Procedure

- 1) **Data Preparation**: Load raw dataset and apply train/validation/test split respecting temporal order.
- 2) **Preprocessing**: Scale using MinMax normalization (fit on training set), then create sequences with the specified lookback.
- 3) **Model Training**: For each combination of (architecture, lookback, hidden units):
 - a) Initialize model with random weights.
 - b) Train on training set using Adam optimizer and MSE loss.
 - c) Monitor validation loss and apply early stopping (patience = 5).
 - d) Restore best weights from validation phase.
- 4) **Evaluation**: Compute MAE and MSE on held-out test set using denormalized predictions.
- 5) **Results Storage**: Record all hyperparameters and metrics to JSON for subsequent analysis.

C. Justification of Methodological Choices

1) *Why Chronological splitting Validation?*: Chronological splitting validation respects the temporal structure of time series data, preventing information leakage from future to past. Random shuffling or standard k-fold cross-validation would violate the fundamental assumption that future values can not influence past values.

2) *Why MinMax Scaling?*: MinMax scaling bounds values to $[0, 1]$, which:

- Improves neural network training stability.
- Allows fair comparison across datasets with different scales.
- Preserves the distribution shape (unlike standardization).

3) *Why Early Stopping with Patience=5?*: This configuration prevents overfitting by stopping when validation loss plateaus while allowing sufficient training. A patience of 5 epochs provides stability without premature termination.

D. Computational Considerations

Total experimental runs: Approximately 1,500+ model trainings (varying by dataset). Each model trains for typically 20–40 epochs before early stopping. All experiments use GPU acceleration where available.

VI. RESULTS AND DISCUSSION

A. Overall Performance Summary

We present comprehensive results across all five datasets, comparing three RNN architectures (Simple RNN, Jordan RNN, Multi-Recurrent RNN) against baseline methods (GRU, LSTM).

TABLE III: Best test MAE achieved for each dataset and architecture

Dataset	Architecture	Test MAE	H. Units/Delay	Lookback
Electricity	GRU	0.0286	128	30
ETTh1	Multi-Recurrent	0.0579	(1, 2, 3)	60
Weather	Multi-Recurrent	0.0011	(1, 6, 12)	36
Traffic	GRU	0.0279	64	36
COVID-19	Jordan	0.1848	128	3

1) *Best Performance by Dataset*: Multi-Recurrent RNN performed best on ETTh1 and Weather, which contain 6 and 21 input features, respectively. GRU performed best on Electricity and Traffic, which contain 321 and 430 input features, respectively. In contrast, Jordan performed best on the sparse COVID-19 dataset, which contains only 365 observations. Overall, the results suggest that model performance varies across datasets, with the best recurrent architecture depending on the characteristics and dimensionality of the data.

B. Electricity Dataset Results

1) *Architecture Comparison*: Among the evaluated architectures, the GRU achieved the best performance with 128 units and a lookback of 30 observations, corresponding to approximately 7.5 hours of historical data. GRUs provide gated memory that can retain relevant information over time while using fewer parameters than LSTMs, making them well suited to capturing recurring consumption patterns without unnecessary model complexity. This likely contributed to their advantage over the simpler RNN, while the additional complexity of LSTMs and multi-recurrent architectures did not provide a corresponding improvement. The Jordan architecture was also less suited to this task than the GRU's direct gated handling of temporal dependencies.

TABLE IV: Electricity dataset: Test MAE for Multi-Recurrent RNN across lookback windows

Lookback	MAE	Lookback	MAE
6	0.0316	30	0.0331
12	0.0300	36	0.0315
18	0.0329	42	0.0315
24	0.0317	48	0.0318

2) *Effect of Lookback Window:* Lookback values between 12–36 perform best, capturing both immediate correlations and daily patterns (electricity consumption has 96 measurements per day at 15-min granularity). Very short windows (6) under-fit, while very long windows (42–48) suffer from overfitting.

C. ETTh1 Dataset Results

The ETT dataset shows clearer patterns due to the transformer heating cycle and lower dimensionality (6 features).

TABLE V: ETTh1: Test MAE for each architecture using optimal hyperparameters

Architecture	Best MAE	Architecture	Best MAE
Simple RNN	0.1199	GRU	0.1082
Jordan RNN	0.1535	LSTM	0.1123
Multi-Rec	0.0857	Improvement	20.7% vs GRU

1) *Architecture Comparison:* The Multi-Recurrent RNN achieved the best performance on the ETTh1 dataset, with a test MAE of 0.0857, representing a 28.4% improvement over the Simple RNN and a 20.7% improvement over the GRU. This suggests that the multiple recurrent pathways were better able to capture the different temporal scales present in the transformer temperature data. In particular, the delayed recurrent connections allow the model to incorporate both short-term changes in temperature and longer-term patterns associated with hourly and daily heating cycles. The GRU and LSTM also benefited from their gated memory mechanisms, achieving MAEs of 0.1082 and 0.1123 respectively, but did not match the Multi-Recurrent architecture. The Jordan RNN performed worst, with an MAE of 0.1535, suggesting that feeding previous outputs back into the network was less effective for capturing the temporal dependencies in this dataset.

D. Weather Dataset Results

The weather dataset, with highest sampling frequency (10 minutes) and clean patterns, showed the smallest absolute errors.

TABLE VI: Weather dataset: Test MAE across architectures and key lookback values

Architecture	Lookback 12	Lookback 24	Lookback 36
Simple RNN (128)	0.0028	0.0016	0.0015
Jordan RNN (128)	0.0030	0.0018	0.0016
Multi-Recurrent (64)	0.0017	0.0012	0.0012
GRU (128)	0.0022	0.0016	0.0015
LSTM (128)	0.0020	0.0014	0.0013

1) *Performance Table by Architecture and Lookback:* The weather dataset exhibits strong periodic behaviour, which makes it well suited to architectures capable of modelling temporal dependencies at multiple time scales. The Multi-Recurrent RNN consistently achieved the lowest MAE across the tested lookback windows, reaching 0.0012 at both 24 and 36 lookback. Its delayed recurrent connections allow the model to incorporate information from different points in the recent history, which is useful for capturing both short-term fluctuations and recurring environmental patterns. The LSTM also performed well, achieving an MAE of 0.0013 at a lookback of 36, while the GRU and Simple RNN achieved 0.0015. The Jordan RNN produced the highest errors across all three lookback values, suggesting that its output-feedback mechanism was less effective for this type of highly periodic data. Overall, the consistently low errors across the larger lookback windows indicate that the weather series benefits from retaining a broader temporal context, with the Multi-Recurrent architecture providing the strongest performance among the evaluated models.

E. Traffic Dataset Results

Traffic forecasting presents particular challenges due to irregular patterns and external events.

1) *Architecture Comparison:* The reported test MAE values for the Traffic dataset are:

TABLE VII: Traffic dataset: Test MAE by architecture

Architecture	Test MAE
GRU	0.0279
Multi-Recurrent	0.0327
LSTM	0.0343
Simple RNN	0.0387
Jordan RNN	0.0369

The GRU achieved the lowest test MAE of 0.0279 using a lookback of 36 and 64 units. This was followed by the Multi-Recurrent RNN with an MAE of 0.0327, while the LSTM, Jordan RNN, and Simple RNN achieved 0.0343, 0.0369, and 0.0387 respectively. The stronger performance of the GRU suggests that its gated memory mechanism was better suited to the irregular temporal dependencies present in traffic data. Unlike highly periodic datasets such as weather or electricity consumption, traffic patterns can vary substantially due to congestion, incidents, weather, and other external factors. The GRU gates allow the model to selectively retain or discard information as these patterns change, providing greater flexibility than the fixed delayed connections of the Multi-Recurrent architecture. The LSTM also provides gated memory, but its additional complexity did not translate into improved performance in this case. The poorer performance of the Simple and Jordan RNNs further suggests that simpler recurrent mechanisms were less effective at handling the variable temporal dependencies in the traffic series.

F. COVID-19 Dataset Results

COVID-19 data presented the most challenging forecasting scenario due to non-stationarity and structural breaks.

TABLE VIII: COVID-19: Best Test MAE for Jordan RNN per lookback values

Lookback	MAE	Lookback	MAE
1	0.3915	6	0.4081
2	0.2621	7	0.4347
3	0.1848	8	0.5838
4	0.4586	9	0.5163
5	0.7425	10	0.8279

1) *Performance Patterns*: The Jordan RNN achieved its lowest MAE of 0.1848 at a lookback of 3, after which performance deteriorated considerably. In particular, increasing the lookback beyond 3 did not provide additional useful historical information, with the MAE increasing to 0.7425 at a lookback of 5 and reaching 0.8279 at a lookback of 10, with similar degrading values up to the lookback of 21. This behaviour is consistent with the highly non-stationary nature of COVID-19 case data, where changes in transmission, interventions, reporting practices, and other external factors can cause historical patterns to become less representative of future observations.

Compared with the other architectures, the Jordan RNN was particularly sensitive to the amount of historical information provided. Its reliance on previous outputs may have allowed it to respond effectively to the very short-term dependencies in the series at a lookback of 3, but the additional historical context appears to have introduced information that was less useful for forecasting. In contrast, architectures such as the GRU and LSTM use gated memory mechanisms to selectively retain relevant information, which can provide greater control over how historical observations influence the prediction. The Multi-Recurrent architecture similarly provides access to information at multiple temporal delays, but the usefulness of these delays is limited when the underlying process contains frequent structural changes. Overall, the results suggest that short-term information was more valuable than long historical windows for this dataset, with the sharp variation in performance across lookback values highlighting the difficulty of applying recurrent models to non-stationary time series.

G. Cross-Dataset Analysis

1) *Architecture Effectiveness*: Overall, the results show that no single recurrent architecture consistently performed best across all datasets. Performance depended strongly on the temporal characteristics of each series. The Multi-Recurrent RNN particularly well on the more periodic ETTh1 and Weather datasets, while the GRU achieved the lowest error on the more irregular Traffic dataset. The Electricity dataset also favoured recurrent models with sufficient temporal memory, with the GRU performing best among the evaluated architectures. The COVID-19 dataset was substantially more difficult, with strong sensitivity to lookback length due to its

non-stationary behaviour and structural changes. These results indicate that model architecture and lookback length should be selected according to the underlying temporal structure of the forecasting problem rather than assuming that one architecture will generalise best across all time series.

TABLE IX: Observed performance by hidden unit count

Hidden Units	Observed Performance
8	Good
16	Good
32	Better
64	Best
128	Good
256	Worse
512	Poor

2) *Hidden Unit Counts*: The results suggest that increasing the number of hidden units does not necessarily improve forecasting performance. Moderate model sizes, particularly 64 and 128 units, generally provided a good balance between model capacity and generalisation, while substantially larger configurations did not consistently produce improvements. Smaller configurations of 8–16 units may have limited capacity to represent more complex temporal dependencies, whereas increasing the number of units beyond the moderate range introduces additional parameters without a corresponding improvement in test performance. The results therefore suggest that moderate hidden-layer sizes are generally sufficient for these datasets, although the optimal number of units remains dependent on the architecture and characteristics of the individual time series.

H. Discussion of Key Findings

1) *Architecture Performance*: The results demonstrate that no single recurrent architecture consistently outperformed the others across all datasets. Instead, performance depended strongly on the temporal characteristics of the forecasting problem. Multi-Recurrent RNNs performed particularly well on the ETTh1 and Weather datasets, which exhibited clearer periodic behaviour and temporal patterns at multiple scales. In contrast, the GRU achieved the lowest reported MAE on the Traffic dataset, suggesting that its gated memory mechanism was better suited to the more irregular temporal dependencies present in that series. The Electricity results also showed that relatively complex recurrent architectures could effectively model consumption patterns, while the COVID-19 results demonstrated substantially greater sensitivity to the choice of lookback window.

2) *Dataset-Dependent Observations*: The results highlight clear differences between datasets. The Weather and ETTh1 datasets exhibited strong periodic patterns and generally benefited from architectures capable of representing temporal dependencies across multiple scales. The Traffic dataset was more irregular, for which the GRU achieved the lowest reported MAE of 0.0279. The Electricity dataset contained substantial temporal structure due to its high-frequency sampling and recurring consumption behaviour, with moderate lookback

windows generally performing well. In contrast, the COVID-19 dataset was considerably more difficult to forecast, with large changes in performance across lookback values reflecting its non-stationary behaviour and structural changes.

3) *Effect of Lookback*: Lookback length had a substantial effect on model performance, although the optimal value varied between datasets. For the Electricity dataset, lookbacks between 12 and 36 observations performed relatively consistently, with the best reported MAE of 0.0300 occurring at 12 observations. For the Weather dataset, performance improved as the lookback increased from 12 to 24–36 observations. In contrast, the COVID-19 dataset favoured a very short lookback, with the Jordan RNN achieving its lowest reported MAE at only 3 observations. These results indicate that the amount of historical information required is strongly dependent on the temporal structure and stationarity of the underlying series.

4) *Practical Implications*: The experiments suggest that recurrent architecture and hyperparameter selection should be treated as dataset-specific decisions rather than assuming a universally optimal model. Architectures with multi-scale temporal connections can be particularly effective for strongly periodic series, while gated architectures such as GRUs may be more suitable when temporal dependencies are irregular. Similarly, hidden-unit count and lookback length should be tuned together with the architecture, as increasing model capacity or historical context does not necessarily improve generalisation. Overall, the results emphasise the importance of matching the model’s temporal structure to the characteristics of the forecasting dataset.

VII. CONCLUSIONS

A. Summary of Findings

This empirical study evaluated five recurrent architectures across five time series datasets with diverse temporal characteristics. The results demonstrate:

- 1) **Architecture performance is dataset-dependent**: no single architecture consistently achieved the lowest test MAE across all datasets.
- 2) **Multi-Recurrent RNNs performed strongly on periodic data**, achieving the lowest reported MAE on the ETTh1 and Weather datasets, while GRU performed best on the Traffic dataset.
- 3) **Model capacity should be moderate**: configurations using 64–128 hidden units generally performed well, while increasing the number of units did not consistently improve generalization.
- 4) **Lookback selection is important**: the optimal historical context varied substantially between datasets, with shorter windows favoured for highly non-stationary data and longer windows benefiting datasets with clearer temporal patterns.

B. Contributions

This work contributes:

- A systematic comparison of Multi-Recurrent RNNs with conventional recurrent architectures on benchmark datasets.
- An empirical investigation of multi-scale temporal modeling in RNNs.
- Analysis of architecture and hyperparameter effects across multiple datasets.
- A time series evaluation methodology designed to avoid temporal information leakage.

C. Limitations and Future Work

1) Limitations:

- The study is limited to single-step-ahead forecasting; multi-step horizons warrant further investigation.
- Fixed delay patterns are used in the Multi-Recurrent RNN; learnable delays could provide additional flexibility.
- Computational costs were not thoroughly analyzed; GRU and LSTM may have different computational characteristics depending on implementation.

2) Future Directions:

- 1) Investigate adaptive delays in Multi-Recurrent RNNs.
- 2) Combine gating mechanisms (GRU) with multi-scale modeling.
- 3) Apply attention mechanisms to weight delay importance.
- 4) Extend the evaluation to multi-step forecasting and probabilistic predictions.
- 5) Evaluate the architectures on additional domains such as finance and speech.

REFERENCES

- [1] M. Tashreeq Hartogh, *RNN Forecasting*, GitHub repository. Available: <https://github.com/calebwidogast96-art/RNN-Forecasting>
- [2] M. Ebadpour, *Recurrent Neural Networks (RNNs) and Implementing Elman and Jordan Architectures*, GitHub repository, 2024. Available: <https://github.com/MohsenEbadpour/Recurrent-Neural-Networks--RNNs--and-implementing-Elman-and-Jordan-architectures>
- [3] Keras, “RNN layer,” *Keras API Documentation*. Available: https://keras.io/api/layers/recurrent_layers/rnn/
- [4] L. Lai, *Multivariate Time Series Data*, GitHub repository. Available: <https://github.com/laiguokun/multivariate-time-series-data>
- [5] A. Trindade, “ElectricityLoadDiagrams20112014,” *UCI Machine Learning Repository*, 2015. Available: <https://archive.ics.uci.edu/dataset/321/electricityloadaddiagrams20112014>
- [6] Max Planck Institute for Biogeochemistry, “Weather Station,” Jena, Germany. Available: <https://www.bgc-jena.mpg.de/wetter/>
- [7] H. Zhou, *ETDataset: Electricity Transformer Dataset*,

GitHub repository.

Available: <https://github.com/zhouhaoyi/ETDataset>

- [8] K. Yi, Q. Zhang, W. Fan, S. Wang, P. Wang, H. He, N. An, D. Lian, L. Cao, and Z. Niu,
“Frequency-domain MLPs are More Effective Learners in Time Series Forecasting,”
in *Proceedings of the Thirty-seventh Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
Available: <https://openreview.net/forum?id=iif9mGCTfy>